Informe final – TPO POO «Patitas»

Nov 10, 2026

Integrantes: Agustin Suarez, Octavio Pena, Palma Santiago · **Propuesta B:** Clínica Veterinaria

Introducción

En la Etapa 2 implementamos en Java el sistema que modelamos en la Etapa 1. Funciona por consola, con un menú para cada rol (recepción, veterinario y administrador), y guarda los datos en archivos de texto para que se conserven entre ejecuciones. Al pasar del diagrama al código tuvimos que cambiar varias cosas del diseño original; en este informe contamos cuáles, por qué, qué problemas tuvimos y cómo trabajamos. El diagrama de clases final está en la última página.

Cambios respecto del diseño original

La estructura general se mantuvo: la jerarquía de mascotas, la historia clínica con sus consultas, el stock por lotes y el patrón Strategy para los descuentos. Los cambios aparecieron al programar, cuando un atributo estaba repetido o una relación complicaba guardar los datos.

En las personas, `Personal` pasó a llamarse `Persona` y ahora `Duenio` también hereda de ella, porque compartían nombre, apellido, documento y teléfono. El teléfono pasó a `String`, porque un celular no entra en un `int`. Del dueño sacamos la lista de mascotas: ahora es la mascota la que conoce a su dueño, lo que simplificó guardar la relación y transferir una mascota. El plan de salud quedó como un `boolean`, porque solo necesitábamos saber si lo tiene.

En `Mascota` agregamos un id, el dueño y la historia clínica como atributos, y el atributo `especie` pasó a ser un método abstracto que cada subclase implementa. Así no se puede crear un perro con especie gato. También agregamos la clase `Conejo`, que el enunciado nombraba y faltaba en el diagrama. En `HistoriaClinica` sumamos `crearConsulta()`, que es la forma concreta de aplicar el patrón Creador que justificamos en la Etapa 1.

Sacamos dos atributos que estaban repetidos. `Insumo` ya no guarda el stock: lo calcula sumando sus lotes no vencidos, así nunca queda desactualizado. `Turno` ya no guarda la duración, porque depende del tipo de atención, que ahora tiene el tiempo y el costo. En `Vacuna`, el intervalo de refuerzo pasó a ser un número de días para poder sumarlo a la fecha de aplicación.

En la factura sacamos la consulta, los tratamientos y las vacunas, que repetían datos: todo lo que se cobra va como línea del detalle, que puede ser un servicio o un producto. A las estrategias de descuento les agregamos `SinDescuento` para el débito, y separamos la impresión en `ImpresionFactura` para que la factura solo calcule. Por último, dividimos `GestorStockFacturacion` en `GestorStock` y `GestorFacturacion`, y agregamos `GestorClientes` para el registro de dueños y mascotas.

Decisiones de implementación

Organizamos el código en los paquetes `model`, `controller` y `view`. Las vistas solo piden datos y muestran resultados, y siempre llaman a un controlador, que coordina el modelo y guarda los cambios. El `Main` carga todos los datos una sola vez y se los pasa a los controladores, para que todas las pantallas trabajen sobre los mismos objetos.

Para la persistencia usamos un repositorio por entidad. La historia clínica se guarda agregando al final del archivo, porque el enunciado dice que solo admite registros nuevos. El resto de los datos se reescribe completo en cada cambio. Entre archivos guardamos identificadores (el código del insumo, el DNI del dueño) y al cargar se busca el objeto real; por eso los archivos se leen en orden, de modo que cada entidad esté cargada antes de que otra la necesite. Las clases con id tienen dos constructores: uno genera un id nuevo y el otro respeta el que estaba guardado.

En las reglas de negocio, el stock se descuenta al finalizar la consulta y no al indicar el tratamiento. Antes de descontar se verifica que alcance para todos los medicamentos, y se consume primero el lote que vence antes. Para dar un turno se valida la seña, el horario del veterinario y que no haya superposición. Una urgencia toma el horario actual del veterinario de guardia y mueve los turnos que se pisan al próximo hueco libre. Como supuesto, un cliente con plan de salud recibe el 15 % aunque pague con crédito, y no se le aplica el recargo.

También usamos los contenidos de la materia que pide el enunciado. `Lote` y `Turno` tienen orden natural con `Comparable`, y para el reporte de stock bajo usamos `ComparadorPorStock`, un `Comparator` que ordena de menor a mayor stock. Creamos cinco excepciones propias para las reglas de negocio (peso no registrado, stock insuficiente, vacuna no aplicable, turno superpuesto y seña requerida), y las vistas las capturan con `try`, `catch` y `finally`.

Dificultades encontradas

Lo que más nos costó fue la persistencia. En el diagrama los objetos se apuntan entre sí, pero en un archivo de texto hay que guardar identificadores y volver a armar las relaciones al leer. Al principio el contador de ids volvía a cero cada vez que abríamos el

programa y se repetían ids; lo resolvimos con el constructor que recibe el id guardado. También tuvimos un problema con los lotes vencidos: al cargarlos, la validación de lote vencido cortaba la lectura, así que separamos el método que agrega un lote nuevo del que lo reconstruye desde el archivo.

Otro problema fue que, al principio, cada pantalla leía los archivos por su cuenta. Un cambio hecho en una pantalla no se veía en otra y las consultas nuevas se perdían. Lo solucionamos cargando todo una vez en el `Main` y compartiendo las listas.

Al juntar el trabajo de todos tuvimos conflictos en los merges. Al reorganizar los paquetes quedaron dos versiones de `Insumo` y de `Lote`, y en un merge se perdió un constructor de `Mascota`. También encontramos errores que no se veían al compilar: el cálculo de minutos de los horarios restaba solo los minutos sin las horas, y había varios `Scanner` leyendo la consola a la vez, lo que hacía que se salteara entradas. Las pruebas unitarias nos ayudaron a encontrar y confirmar estos arreglos.

Distribución de tareas

Dividimos el trabajo por área del sistema: personas y mascotas, atención e historia clínica, turnos, stock, facturación, persistencia, y pruebas y documentación. Cada uno trabajó en su parte y la integramos en el repositorio con merges, resolviendo los conflictos que aparecieron. El aporte de cada integrante queda registrado en el historial de commits.

Reflexión del grupo

El diagrama de la Etapa 1 nos sirvió como punto de partida, pero muchos errores aparecieron recién al programar. Si empezáramos de nuevo, pensaríamos desde el principio cómo se va a guardar cada clase. Los patrones que elegimos sí resultaron útiles: con el Strategy, agregar un medio de pago fue crear una clase, y separar repositorios y controladores nos permitió cambiar los archivos sin tocar el modelo. También aprendimos que hay que integrar seguido: cuando trabajamos mucho tiempo por separado, después nos costó juntar todo.

Quedó pendiente guardar las facturas en archivo, anularlas con nota de crédito y registrar notas rectificatorias desde el menú. Las clases están, pero no llegamos a conectarlas.

Anexo: diagrama de clases final

